Retrieval-Augmented Generation (RAG)

A Basic Introduction to Enhancing LLMs with External Knowledge

Welcome to this exploration of how retrieval mechanisms can dramatically improve the capabilities and reliability of large language model workflows. Today, we'll discover why RAG has become a cornerstone technique in modern NLP system design.

The Fundamental Limitations of LLMs

While large language models have revolutionized natural language processing, they face several critical constraints that limit their real-world deployment:

Knowledge Cutoff

Training data frozen at a specific date means models lack awareness of recent events, updates, or changes in information.

Hallucination Risk

Models can generate plausible-sounding but factually incorrect information with confidence.

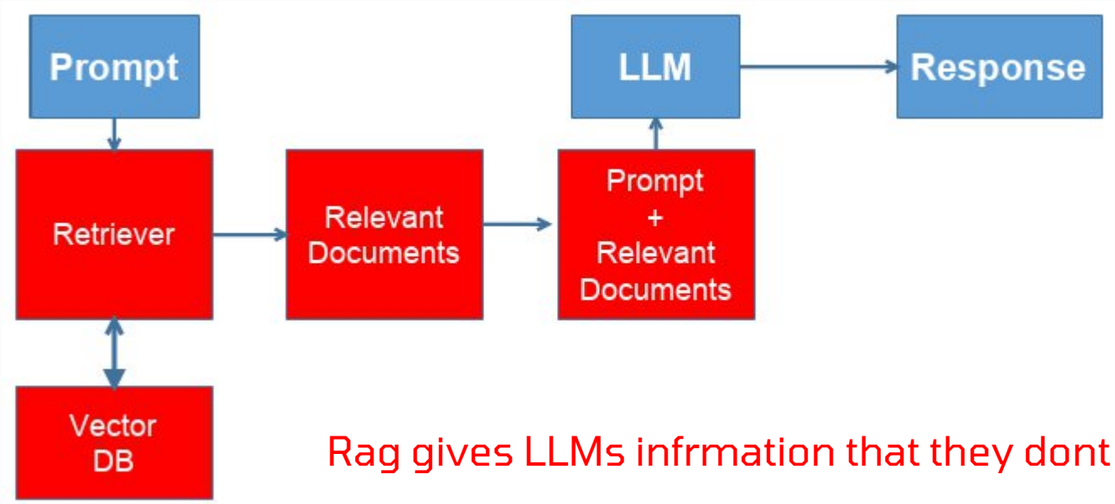
Context Window Costs

Processing large context windows is computationally expensive and increases inference latency significantly.

Domain Knowledge Gaps

Models have no access to private corporate data, proprietary documents, or specialized domain knowledge not present in training data.

What is Retrieval-Augmented Generation?



Rag gives LLMs infrmation that they dont know from their training

> A system that **retrieves relevant information** from a knowledge store and **augments** an LLM with it during generation.

Critical Distinction: RAG is *not* fine-tuning an LLM. Instead, it dynamically adds relevant external information to the model's prompt at inference time, allowing the model to ground its responses in retrieved facts.

This approach separates the model's parametric knowledge (information stored in the models weights) from a user selected knowledge base. **It enables updates without retraining and offers transparency into the sources used for LLM generation.**

image derived from https://learn.deeplearning.ai/courses/retrieval-augmented-generation

Core Components of a RAG System

01

Knowledge Base / Document Store

A collection of documents, passages, or structured data that serves as the external memory. This could be company wikis, research papers, product documentation, or any domain-specific corpus.

02

Embedding Model + Vector Index

A neural encoder that transforms text into dense vector representations, stored in a searchable index structure optimized for similarity search.

03

Retrieval Engine (Top-k Search)

The mechanism that performs **fast** similarity search to find the k most relevant documents or passages given a query embedding.

04

Prompt Construction + LLM Generation

The process of assembling retrieved contexts with the user query into a structured prompt, then feeding it to the language model for final answer generation.

RAG Architecture Flow

Each step in this pipeline is critical.

- **The query must be embedded using the same model as the documents**
- **The search must return truly relevant contexts**
- **The prompt must effectively guide the LLM to use the retrieved information appropriately.**

Understanding Embeddings

Key Concepts

Dense Vector Mapping

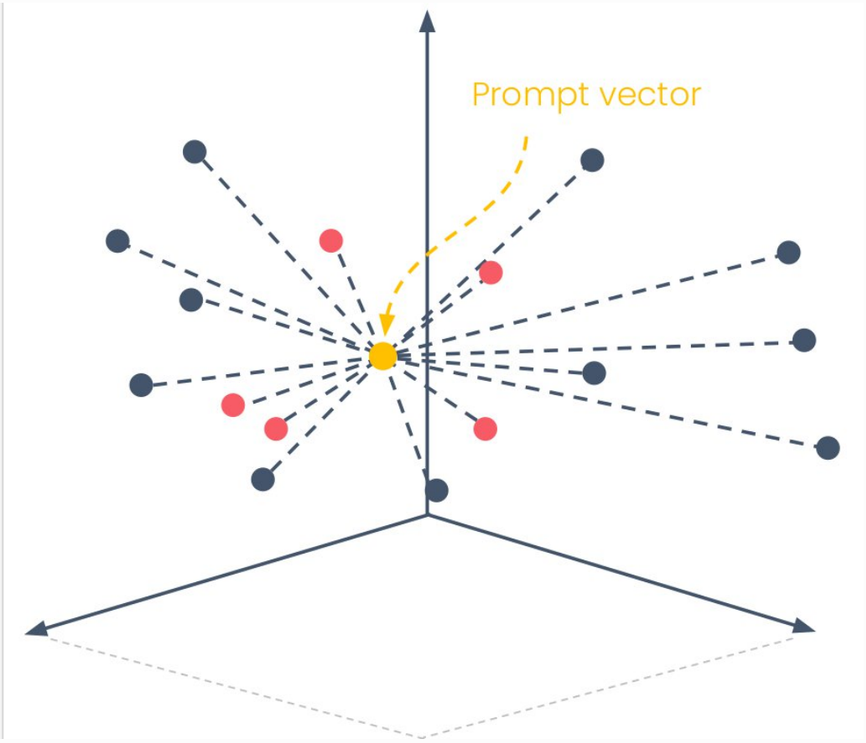
Text is transformed into high-dimensional vectors (typically 384-1536 dimensions) that capture semantic meaning.

Semantic Similarity

Text with similar meanings produces vectors that are close together in the embedding space, regardless of exact word overlap.

Fast Similarity Search

Enables efficient retrieval via cosine similarity or dot product operations on millions of vectors in milliseconds.



Embeddings leverage the encoder portion of transformer architectures, producing contextualized representations where each token's vector depends on its surrounding context.

image derived from https://learn.deeplearning.ai/courses/retrieval-augmented-generation

Vector Database Technology

Modern RAG systems rely on specialized vector databases optimized for high-dimensional similarity search. These tools provide the infrastructure for fast, scalable retrieval. **We will talk about how these work next.**

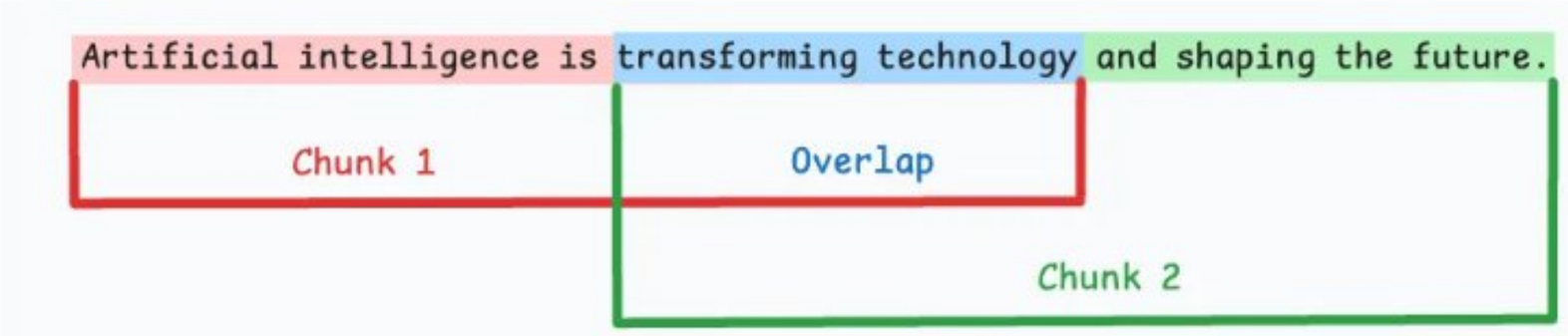
Facebook's library for efficient similarity search, offering multiple index types and GPU acceleration for massive-scale applications.

Developer-friendly open-source embedding database designed for easy integration with Python applications.

Managed cloud vector database with automatic scaling and low-latency search capabilities.

Open-source vector database built for production deployments with distributed architecture support.

Key Trade-offs: Query speed versus index size and memory requirements. Approximate nearest neighbor algorithms sacrifice perfect accuracy for dramatic speed improvements. **We will study this algorithm next week**



Document Chunking Strategy

Why Chunking Matters

Large documents contain multiple concepts and topics. Embedding an entire document as a single vector will produce a representation that's too general to match specific queries effectively. The solution is to break documents into smaller, focused chunks. **There are many chunking strategies, the most approachable is fixed size.**

Documents Too Long

Full documents exceed optimal embedding length and **dilute semantic signal with multiple topics.**

Split into Chunks

Break into smaller segments (256-512 tokens typical). Each chunk ideally represents one coherent concept.

Add Overlap

Include 10-20% overlap between consecutive chunks to prevent information loss at boundaries.

Advanced Strategies

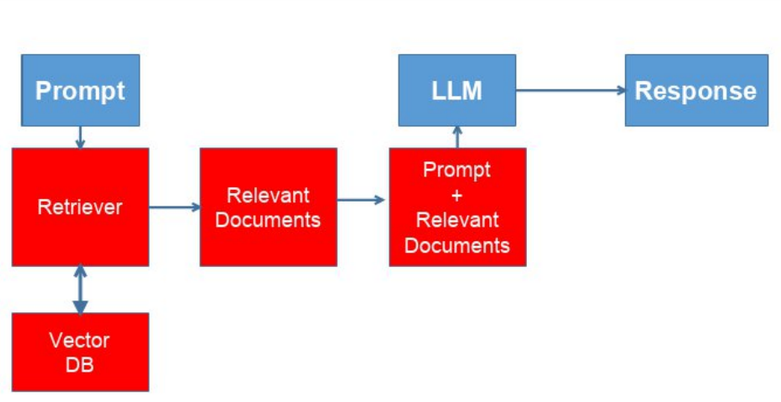
Semantic chunking using LLMs to identify natural topic boundaries produces better results than fixed-size splits.

image derived from https://blog.dailydoseofds.com/p/5-chunking-strategies-for-rag

Constructing Effective Prompts

The Assembly Process

Once relevant documents are retrieved, they must be integrated into the LLM prompt in a structured way that guides the model to use them effectively.



Example Template Structure BTW many models have different prompt template requirements :(

```
You are a helpful assistant. Answer the user's question **using only** the provided
context.
If the answer isn't in the context, say you don't know.
Question: {user_question}
Context:
{context}
Instructions:
- Ground your answer in the context.
- If the context is insufficient, say "I don't know based on the provided context."
- Cite the chunk indices you used (e.g., [CHUNK 0, CHUNK 2]).
```

This simple template provides clear sections for context and user question. The model learns to reference the provided context when generating answers.

Critical Considerations

- Keep total prompt length under the model's context window (typically 2K-8K tokens)
- Place most relevant retrieved documents first or last (recency bias)
- Add instructions like "Answer based only on the provided context"
- Consider including source citations in the template

Real-World Use Cases for RAG

Q&A Over Private Company Docs

Enable employees to query internal documentation, policies, procedures, and knowledge bases using natural language. Reduces time spent searching and improves information accessibility across organizations.

Chatbots with Up-to-Date Knowledge

Build customer service bots that access current product information, troubleshooting guides, and FAQ databases. Update the knowledge base without retraining the underlying model.

Domain-Specific Assistants

Deploy specialized assistants in legal, medical, financial, or technical domains. Ground responses in authoritative sources like case law, medical literature, or regulatory documents to ensure accuracy.

Long-Form Document Summarization

Summarize lengthy reports, research papers, or document collections by retrieving relevant sections and synthesizing them into coherent summaries that respect the original content.

RAG vs Fine-Tuning: A Comparison

Both approaches enhance LLM capabilities for specific domains, but they involve fundamentally different trade-offs in terms of flexibility, cost, and maintenance.

RAG Strengths

- Fast to update—just add new documents
- Transparent sourcing and explainability
- Access to external, current knowledge
- Lower upfront training costs

RAG Weaknesses

- Retrieval errors propagate to generation
- Added latency from search step
- Dependency on retrieval quality

Fine-Tuning Strengths

- Knowledge embedded directly in model
- Better task-specific performance
- No retrieval latency
- Consistent behavior patterns

Fine-Tuning Weaknesses

- Expensive computational requirements
- Static knowledge after training
- Requires retraining for updates
- Risk of catastrophic forgetting

Fundamental Limitations of RAG

While RAG systems offer powerful capabilities, they introduce new failure modes and challenges that must be understood and addressed in production deployments.

1

Retrieval Noise

The search mechanism may return irrelevant or tangentially related documents, introducing misleading context that degrades answer quality. This is especially problematic with ambiguous queries.

2

Persistent Hallucinations

Even with retrieved documents, LLMs can still hallucinate if the provided context is incomplete, contradictory, or unclear. The model may blend retrieved facts with generated fiction.

3

Index Maintenance Costs

Keeping the vector index synchronized with source documents requires continuous re-embedding and indexing. For large, frequently updated corpora, this becomes a significant operational burden.

4

Context Window Constraints

Retrieved documents consume valuable context window space. With limited windows (2K-8K tokens), there's a trade-off between the number of retrieved documents and the complexity of the query or response.

Summary and Next Steps

Key Takeaways from Today

RAG = Retrieve + Generate

RAG systems augment LLMs with external knowledge through a two-stage process: retrieving relevant information and incorporating it into generation.

Embeddings + Vector Search

Dense embeddings and similarity search form the foundation of efficient retrieval at scale, enabling semantic matching beyond keyword overlap.

Context Through Prompting

Retrieved documents are injected into prompts as context, guiding the LLM to produce grounded, factual responses based on external sources.

Trade-offs Matter

RAG offers different advantages compared to fine-tuning: faster updates and external knowledge access versus embedded, task-specific performance.

Looking Ahead: Next Lecture

In future sessions, we'll dive deeper into **vector search algorithms** such as ANN and HNSW, **advanced chunking strategies**, explore **Weaviate and Chroma** vector databases in detail, learn how to evaluate retrieval quality using **precision, recall, and MRR metrics**.

Preparation: Install Chroma and select a sample document collection (10-50 docs) to experiment with different chunking approaches.